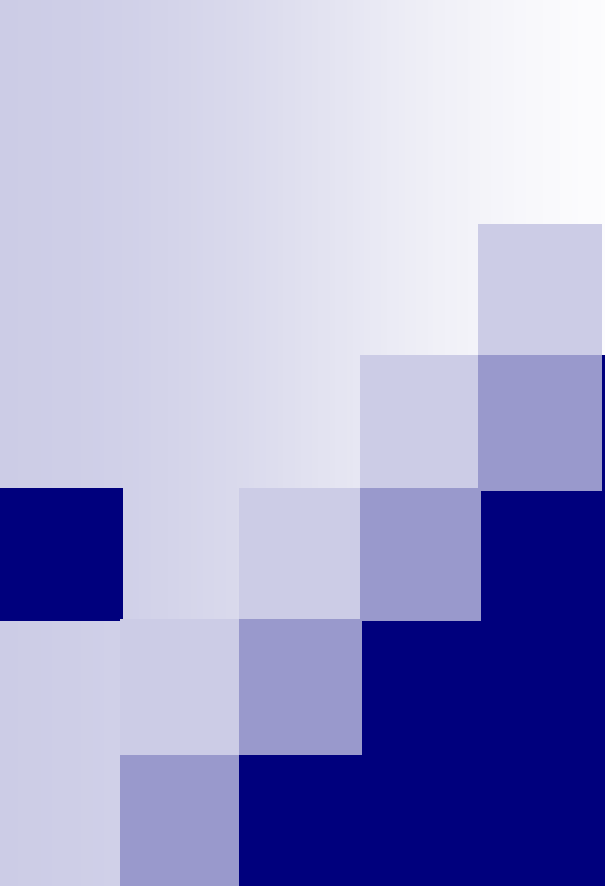




Quality Design Concepts – II



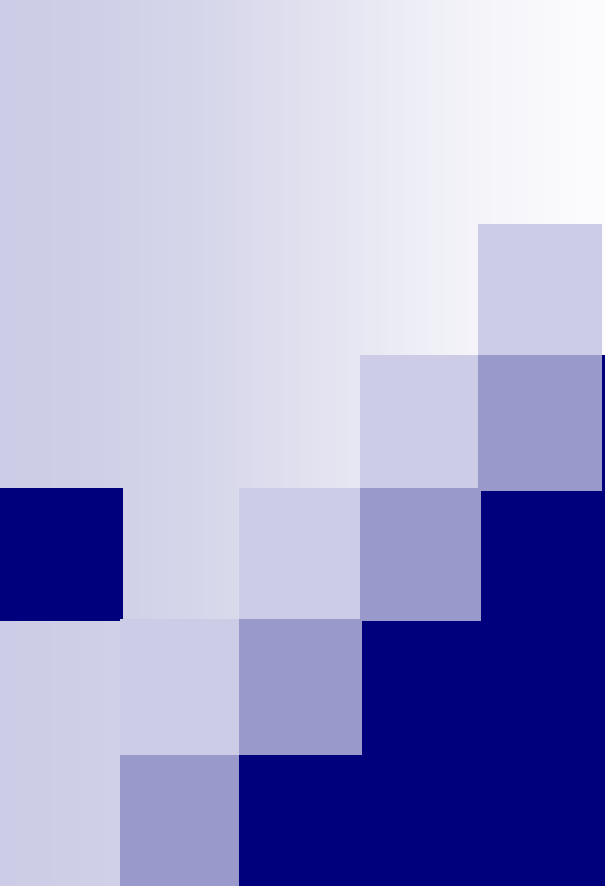
Lecture # 12

Recap

- We were discussing quality design concepts in the last lecture. We had discussed abstraction, refinement, and modularity
- Before that we had discussed the design principles and design evaluation guidelines for software design process and software design model

Today's Lecture

- Continue with our discussion on high quality attributes for design model
- We'll also discuss characteristics of software design methods, without discussing any specific design method



Quality Design Concepts



Information Hiding - 1

- Modules should be specified and designed so that information (procedures and data) contained within a module is inaccessible to other modules that have no need for such information
- IH means that effective modularity can be achieved by defining a set of independent modules that communicate with one another only that information necessary to achieve a software function

Information Hiding - 2

- Abstraction helps to define the procedural (or informational) entities that make up the software
- IH defines and enforces access constraints to both procedural detail within a module and any local data structure used by the module

Information Hiding - 3

- The greatest benefits of IH are achieved when modifications are required during testing and later, during software maintenance
- Both data and procedures are hidden from other parts of the software, inadvertent errors introduced modifications are less likely to propagate to other locations within the software

Cohesion - 1

- Cohesion is the qualitative indication of the degree to which a module focuses on just one thing
- In other words, cohesion is a measure of the relative functional strength of a module
- A cohesive module performs one single task or is focused on one thing

Cohesion - 2

- Highly cohesive modules are better, however, mid-range cohesion is acceptable
- Low-end cohesion is very bad

Types of Cohesion - 1

- Coincidental cohesion occurs when unrelated components of a system are bundled together
- Logical cohesion occurs when components performing similar functions are put together (e.g., error handling)
- Temporal cohesion occurs when components that are activated at a single time or time span are put together

Types of Cohesion - 2

- Procedural cohesion occurs when the elements in a component form a single control sequence
- Communicational cohesion occurs when all elements of a component operate on the same input data, produce the same output data, or operate on one area of a data structure

Types of Cohesion - 3

- Sequential cohesion occurs when the output of one element in a component serves an input for another element
- Functional cohesion occurs when each part of a component is necessary for the execution of a single function

Types of Cohesion - 4

- Object cohesion occurs when the functions of a common class of objects are aggregated with the class. In this scheme, interfaces are specified to reveal as little as possible about the inner workings of the object

Coupling

- Coupling is a qualitative indication of the degree to which a module is connected to other modules and to the outside world
- In other words, coupling is a measure of interconnection among modules in a software structure
- Loose coupling is better. Simple connectivity is easier to understand and less prone to “ripple effect”

Type of Coupling - 1

- Indirect coupling occurs when one object interacts with another object through a third component, usually a controller. Another form of indirect coupling occurs when using a data-driven style of computation
- Data coupling occurs when a portion of a data structure is passed among modules. When using data coupling, shorter parameter lists are preferred to longer ones

Type of Coupling - 2

- Stamp coupling occurs when a portion of data structure is passed among modules. When using stamp coupling, a smaller number of actual arguments are preferable to a larger number because the only data that should be passed to a module is what it requires

Type of Coupling - 3

- Control coupling occurs when information is passed to a module that affects its internal control. This is undesirable because it requires the calling module to know the internal operation of the module being called
- External coupling occurs when a module depends on the external environment

Type of Coupling - 4

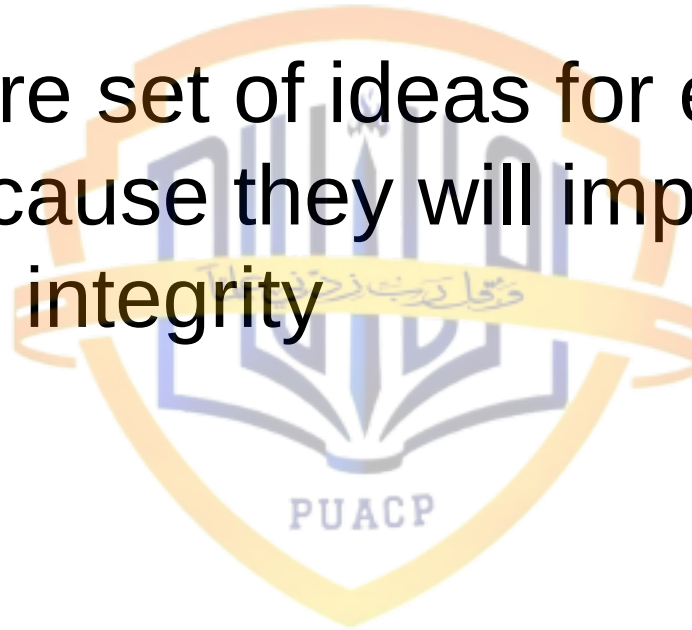
- Common coupling occurs when modules access common areas of global or shared data. This form of coupling can cause one module to unintentionally interfere with the operation of another module
- Content coupling occurs when one module uses information contained in another module

Other Design Concepts

- Software Architecture
- Control Hierarchy
- Structured Partitioning
- Data Structures
- Software Procedure
- Functional Independence
- Encapsulation
- Inheritance
- Polymorphism

A Word of Advice

- Adopt a core set of ideas for each system design, because they will improve their conceptual integrity

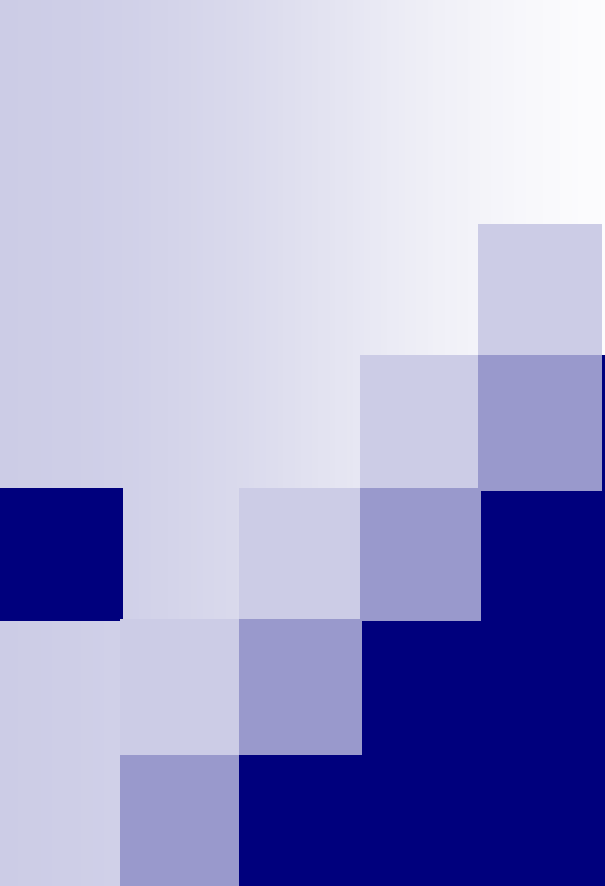


Design Methods

- Use a design method, which is most suitable for the problem at hand. Don't just use the latest or the most popular design method
- There are many structured design and object-oriented design methods to choose from
- Follow the design method's representation scheme. It helps in understanding design

Characteristics of Design Methods

- A mechanism for the translation of analysis model into design representation
- A notation for representing functional components and their interfaces
- Heuristics for refinement and partitioning
- Guidelines for quality assessment



Effective Modularity Criteria for Design Methods

B. Meyer

- 
- Modular decomposability
 - Modular composability
 - Modular understanding
 - Modular continuity
 - Modular protection
- 

Modular Decomposability

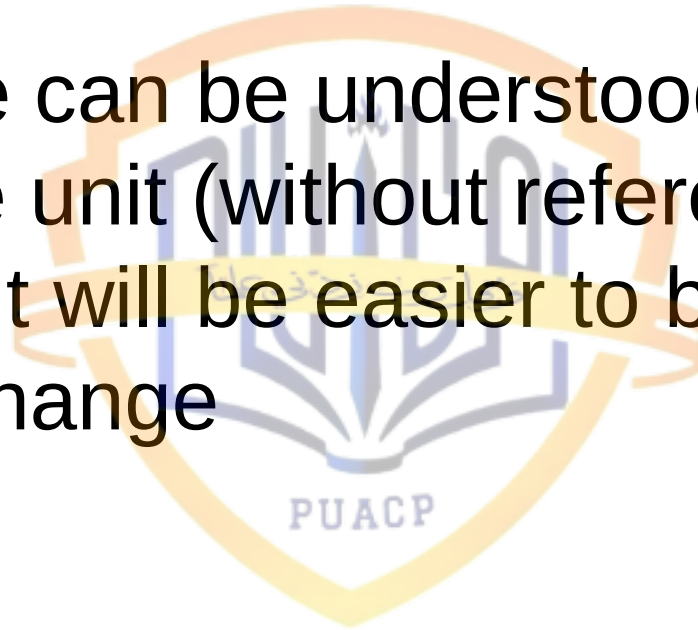
- If a design method provides a systematic mechanism for decomposing the problem into subproblems, it will reduce the complexity of the overall problem

Modular Composability

- If a design method enables existing (reusable) design components to be assembled into a new system, it will yield a modular solution that does not reinvent the wheel

Modular Understandability

- If a module can be understood as a standalone unit (without reference to other modules), it will be easier to build and easier to change



Modular Continuity

- If small changes to the system requirements result in changes to individual modules, rather than system-wide changes, the impact of change-induced side effects will be minimized

Modular Protection

- If an aberrant condition occurs within a module and its effects are constrained within that module, the impact of error-induced side effects will be minimized

Summary

- We have talked about design concepts, which can be considered as quality attributes of software designs
- In the last three lectures, we have discussed software design and how it relates to software quality
- Design is the process where quality is fostered

References

- Software Engineering: A Practitioner's Approach by Roger Pressman (Chapter 13)

